

Nguyễn Anh Khoa

23127391

CI/CD & Test-Harness Engineering

Mục lục

- Phần 1: Tổng quan về CI/CD (Continuous Integration & Continuous Delivery/Deployment)**
 - 1.1 Khái niệm cơ bản (CI vs. CD vs. CD)
 - 1.2 Vai trò và Lợi ích trong Phát triển Phần mềm Hiện đại
 - 1.3 Quy trình hoạt động tiêu chuẩn của một CI/CD Pipeline
- Phần 2: Tổng quan về Test-Harness Engineering**
 - 2.1 Định nghĩa Test Harness và Test-Harness Engineering
 - 2.2 Sự khác biệt giữa Test Harness, Test Framework và Test Suite
 - 2.3 Các thành phần cốt lõi của một Test Harness tiêu chuẩn
 - 2.4 Tầm quan trọng của Test Harness trong CI/CD
- Phần 3: Công cụ hỗ trợ CI/CD (CI/CD Tools)**
 - 3.1 Nhóm Công cụ Self-Hosted & Enterprise (Jenkins, GitLab CI/CD, Bamboo)
 - 3.2 Nhóm Dịch vụ Cloud-Managed (GitHub Actions, CircleCI, Travis CI)
 - 3.3 Nhóm Công cụ GitOps & Cloud-Native (ArgoCD, FluxCD)
 - 3.4 Bảng so sánh các công cụ CI/CD phổ biến
- Phần 4: Công cụ hỗ trợ Test-Harness Engineering**
 - 4.1 Unit Testing Frameworks
 - 4.2 Integration & API Testing Tools
 - 4.3 End-to-End (E2E) Testing Frameworks
 - 4.4 Mocking, Stubbing & Service Virtualization
 - 4.5 Performance & Load Testing Tools
 - 4.6 Test Reporting & Analytics Dashboards
- Phần 5: Chiến lược tích hợp Test-Harness vào CI/CD Pipeline**
 - 5.1 Áp dụng Kim tự tháp Kiểm thử (Test Pyramid) vào Pipeline
 - 5.2 Kỹ thuật chạy kiểm thử song song (Test Parallelization)
 - 5.3 Xử lý lỗi không ổn định (Flaky Tests)
 - 5.4 Shift-Left & Shift-Right Testing
- Phần 6: Xu hướng tương lai của CI/CD & Test-Harness Engineering**
 - 6.1 Môi trường kiểm thử tạm thời (Ephemeral Testing Environments)
 - 6.2 Ứng dụng AI/ML trong Kiểm thử tự động (AI-Driven Testing)

Phần 1: Tổng quan về CI/CD (Continuous Integration & Continuous Delivery/Deployment)

1.1 Khái niệm cơ bản

- Continuous Integration (CI - Tích hợp liên tục):** Là thực hành phát triển phần mềm nơi các lập trình viên thường xuyên gộp (merge) các thay đổi mã nguồn của họ vào một nhánh chính (main branch), lý tưởng nhất là nhiều lần trong ngày. Mỗi lần gộp sẽ tự động kích hoạt quá trình build và chạy test suite để phát hiện lỗi tích hợp càng sớm càng tốt.
- Continuous Delivery (CD - Chuyển giao liên tục):** Tiếp nối sau CI, đảm bảo rằng mã nguồn sau khi build và test thành công luôn ở trạng thái sẵn sàng để triển khai lên bất kỳ môi trường nào (Staging, UAT, Production). Quá trình đẩy code lên môi trường production thường yêu cầu một bước phê duyệt thủ công.
- Continuous Deployment (CD - Triển khai liên tục):** Đi xa hơn một bước so với Continuous Delivery. Mọi thay đổi vượt qua tất cả các giai đoạn của pipeline kiểm thử sẽ tự động được triển khai trực tiếp lên môi trường Production mà không cần sự can thiệp thủ công.

1.2 Vai trò và Lợi ích trong Phát triển Phần mềm Hiện đại

- Phát hiện lỗi sớm:** Tích hợp liên tục buộc mã nguồn phải được kiểm tra qua các bài test tự động ngay khi commit, giúp lập trình viên cô lập lỗi dễ dàng hơn.
- Giảm thiểu rủi ro tích hợp:** Việc tích hợp code thường xuyên loại bỏ tình trạng xung đột mã nguồn nghiêm trọng xảy ra khi nhiều lập trình viên làm việc độc lập trong thời gian dài.
- Rút ngắn thời gian đưa sản phẩm ra thị trường:** Tự động hóa các bước build, test, deploy giúp đẩy nhanh tốc độ phân phối tính năng mới tới người dùng cuối.
- Tối ưu hóa hiệu suất:** Giải phóng đội ngũ phát triển và vận hành khỏi các tác vụ thủ công lặp đi lặp lại.

1.3 Quy trình hoạt động tiêu chuẩn của một CI/CD Pipeline

Một pipeline điển hình bao gồm các giai đoạn sau:

- Commit:** Lập trình viên đẩy code mới lên hệ thống kiểm soát phiên bản (như GitHub, GitLab).
- Build:** Hệ thống CI tải mã nguồn, biên dịch (nếu có), giải quyết các dependencies và đóng gói ứng dụng (thành Docker image, file zip, jar, v.v.).
- Unit & Integration Test:** Chạy các bài kiểm thử đơn vị và kiểm thử tích hợp cơ bản để đảm bảo logic xử lý đúng đắn.

4. **Static Code Analysis:** Quét mã nguồn để tìm kiếm lỗi hỏng bảo mật, lỗi cú pháp và kiểm tra tiêu chuẩn code (Linting, SonarQube).
5. **Deploy to Staging:** Triển khai phiên bản vừa build lên môi trường giả lập (Staging/UAT).
6. **Acceptance & E2E Testing:** Thực thi các kịch bản kiểm thử giao diện (UI) hoặc kiểm thử luồng người dùng hoàn chỉnh (End-to-End) trên môi trường Staging.
7. **Deploy to Production:** Triển khai lên môi trường thực tế (tự động hoặc thông qua phê duyệt).
8. **Monitoring & Alerting:** Giám sát hoạt động của hệ thống sau triển khai để phát hiện lỗi phát sinh trên môi trường thực tế.

Phần 2: Tổng quan về Test-Harness Engineering

2.1 Định nghĩa Test Harness và Test-Harness Engineering

- **Test Harness:** Là một tập hợp các công cụ, thư viện phần mềm, dữ liệu kiểm thử và cấu hình được thiết kế để tự động hóa việc thực thi các ca kiểm thử (test cases), thu thập kết quả và báo cáo. Nó đóng vai trò như một "bộ đỡ" cho phép thực hiện các thử nghiệm kiểm thử dưới những điều kiện được kiểm soát chặt chẽ.
- **Test-Harness Engineering:** Là chuyên ngành kỹ thuật tập trung vào việc thiết kế, xây dựng, bảo trì và tối ưu hóa hệ thống Test Harness này. Kỹ sư Test-Harness chịu trách nhiệm đảm bảo hệ thống kiểm thử chạy nhanh, ổn định, dễ mở rộng và tích hợp mượt mà vào quy trình CI/CD.

2.2 Sự khác biệt giữa Test Harness, Test Framework và Test Suite

Khái niệm	Định nghĩa chính	Ví dụ
Test Suite	Tập hợp các ca kiểm thử (test cases) cụ thể nhằm xác minh một chức năng hoặc một phần của hệ thống.	Bộ test cases kiểm tra tính năng thanh toán bằng thẻ tín dụng.
Test Framework	Tập hợp các quy tắc, chuẩn mực, thư viện cú pháp giúp lập trình viên viết code test một cách có cấu trúc.	Jest, PyTest, JUnit, Playwright.
Test Harness	Toàn bộ môi trường thực thi, bao gồm test framework, dữ liệu giả lập (mocks/stubs), cấu hình hạ tầng, trình sinh báo cáo và công cụ kết nối với CI/CD.	Một hệ thống Dockerized tự động chạy Playwright tests, kết nối với WireMock để giả lập API bên thứ ba, và xuất kết quả ra Allure Dashboard.

2.3 Các thành phần của một Test Harness tiêu chuẩn

- **Test Execution Engine:** Trình khởi chạy (runner) có nhiệm vụ đọc các file test, phân bổ tài nguyên phần cứng, điều phối việc chạy song song (parallelization) và thực thi các ca kiểm thử.
- **Test Data Management (TDM):** Cơ chế chuẩn bị dữ liệu đầu vào cho các test case (seeding database, đọc file cấu hình JSON/CSV) và dọn dẹp dữ liệu sau khi kiểm thử kết thúc (cleanup/teardown).
- **Mocking & Service Virtualization:** Bộ giả lập các thành phần phụ thuộc ngoại vi (như cổng thanh toán, dịch vụ gửi SMS) để đảm bảo bài test không bị ảnh hưởng bởi các yếu tố bên ngoài.
- **Result Collectors & Reporting Engine:** Thành phần ghi lại trạng thái (Pass/Fail), thu thập logs, chụp ảnh màn hình (screenshot), quay video khi kiểm thử thất bại và chuyển đổi sang các định dạng báo cáo tiêu chuẩn (JUnit XML, HTML).

2.4 Tầm quan trọng của Test Harness trong CI/CD

Không có một Test Harness vững chắc, CI/CD pipeline chỉ là một vỏ bọc rỗng. Test Harness cung cấp tính tin cậy cho pipeline bằng cách:

- Ngăn chặn việc đưa mã nguồn lỗi lên Production.
- Đảm bảo các bài kiểm thử chạy nhất quán giữa môi trường máy cục bộ (Local) và máy chủ CI (CI Agent).
- Tối ưu hóa thời gian phản hồi thông qua việc quản lý tài nguyên chạy test hiệu quả.

Phần 3: Công cụ hỗ trợ CI/CD (CI/CD Tools)

3.1 Nhóm Công cụ Self-Hosted & Enterprise

Các công cụ này yêu cầu tổ chức tự cài đặt, vận hành và quản lý hạ tầng máy chủ của riêng mình.

1. Jenkins

- **Tổng quan:** Công cụ CI/CD mã nguồn mở lâu đời và phổ biến nhất thế giới. Phát triển bằng Java, dựa trên hệ sinh thái plugin cực kỳ đồ sộ (hơn 1800 plugins).
- **Cơ chế hoạt động:** Sử dụng `Jenkinsfile` với cú pháp Declarative hoặc Scripted Pipeline dựa trên ngôn ngữ Groovy.
- **Ưu điểm:** Miễn phí, tùy biến không giới hạn, cộng đồng hỗ trợ khổng lồ, phù hợp cho mọi loại kiến trúc từ cổ điển đến hiện đại.
- **Nhược điểm:** Chi phí vận hành cao ("Jenkins maintenance fatigue"), giao diện cũ kỹ, đòi hỏi kỹ năng cấu hình hệ thống sâu.

2. GitLab CI/CD

- **Tổng quan:** Công cụ CI/CD tích hợp sẵn bên trong nền tảng quản lý mã nguồn GitLab.
- **Cơ chế hoạt động:** Định nghĩa pipeline qua file `.gitlab-ci.yml`. Sử dụng các GitLab Runner (viết bằng Go) để thực thi các jobs trên các môi trường khác nhau (Docker, Kubernetes, Shell).
- **Ưu điểm:** Tích hợp cực kỳ chặt chẽ với Git repository, Merge Request và Container Registry. Hỗ trợ tốt cho Kubernetes. Giao diện trực quan, dễ cấu hình.
- **Nhược điểm:** Phiên bản Enterprise có chi phí khá cao đối với các tổ chức lớn.

3. Bamboo (Atlassian)

- **Tổng quan:** Giải pháp CI/CD thương mại thuộc hệ sinh thái của Atlassian.
- **Ưu điểm:** Tích hợp hoàn hảo với Jira, Bitbucket và Confluence. Khả năng quản lý release và phân quyền doanh nghiệp rất tốt.
- **Nhược điểm:** Phí bản quyền đắt đỏ, hệ sinh thái đóng kín hơn so với Jenkins hay GitLab.

3.2 Nhóm Dịch vụ Cloud-Managed

Các giải pháp SaaS (Software-as-a-Service) nơi nhà cung cấp quản lý toàn bộ hạ tầng thực thi pipeline.

1. GitHub Actions

- **Tổng quan:** Nền tảng CI/CD được tích hợp trực tiếp vào GitHub, phát triển nhanh chóng nhờ hệ sinh thái GitHub Marketplace.
- **Cơ chế hoạt động:** Định nghĩa workflow bằng file YAML nằm trong thư mục `.github/workflows/`. Thực thi trên các GitHub-hosted runners (Ubuntu, Windows, macOS) hoặc Self-hosted runners.
- **Ưu điểm:** Dễ tiếp cận, miễn phí cho các kho lưu trữ public (và có hạn mức miễn phí lớn cho private), thư viện Actions có sẵn vô cùng phong phú giúp giảm thiểu việc viết script thủ công.
- **Nhược điểm:** Giới hạn thời gian chạy và băng thông ở các gói tài khoản miễn phí; việc debug pipeline đôi khi khó khăn hơn so với môi trường local.

2. CircleCI

- **Tổng quan:** Dịch vụ CI/CD đám mây tập trung vào tốc độ và hiệu năng thực thi.
- **Cơ chế hoạt động:** Sử dụng tệp cấu hình `.circleci/config.yml`.
- **Ưu điểm:** Khả năng tối ưu hóa bộ nhớ đệm (caching) và chạy song song (parallelism) cực tốt, giúp tăng tốc độ build đáng kể. Hỗ trợ tốt các cấu hình Docker phức tạp.
- **Nhược điểm:** Chi phí tăng nhanh theo mức độ sử dụng tài nguyên (resource class).

3. Cloud Provider Native Tools (AWS CodePipeline, Azure Pipelines, Google Cloud Build)

- **Tổng quan:** Các công cụ CI/CD được phát triển bởi các gã khổng lồ đám mây để phục vụ riêng cho hệ sinh thái của họ.
- **Ưu điểm:** Tối ưu hóa bảo mật và quyền truy cập (IAM), tích hợp mượt mà với các dịch vụ serverless hoặc Kubernetes của chính đám mây đó (như AWS ECS/EKS, Azure AKS).

3.3 Nhóm Công cụ GitOps & Cloud-Native (CD chuyên biệt)

Với sự trỗi dậy của Kubernetes, mô hình GitOps ra đời nhằm tách biệt CI và CD. Mã nguồn sau khi build thành công ở bước CI sẽ được cập nhật phiên bản (tag) vào một Git repository quản lý cấu hình hạ tầng. Các công cụ GitOps sẽ theo dõi repository này để đồng bộ hóa trạng thái trên cluster Kubernetes.

1. ArgoCD

- **Tổng quan:** Công cụ GitOps declarative liên tục phân phối (CD) dành riêng cho Kubernetes.
- **Cơ chế hoạt động:** Hoạt động theo mô hình Pull-based. ArgoCD chạy bên trong Kubernetes cluster, liên tục so sánh trạng thái mong muốn (định nghĩa trong Git) với trạng thái thực tế của cluster. Nếu phát hiện sai lệch (drift), nó sẽ tự động đồng bộ hóa (sync).
- **Ưu điểm:** Giao diện trực quan đẹp mắt, quản lý trạng thái tài nguyên Kubernetes rất tốt, bảo mật cao (không cần cấp quyền deploy từ ngoài vào cluster).
- **Nhược điểm:** Chỉ giới hạn cho các ứng dụng chạy trên Kubernetes.

2. FluxCD

- **Tổng quan:** Một công cụ GitOps mã nguồn mở khác thuộc CNCF (Cloud Native Computing Foundation).
- **Ưu điểm:** Nhẹ nhàng, thiết kế theo triết lý modular, tích hợp rất tốt với các công cụ bảo mật và Kubernetes API gốc.
- **Nhược điểm:** Không có giao diện quản lý đồ họa (UI) chính thức mạnh mẽ như ArgoCD (chủ yếu thao tác qua CLI).

3.4 Bảng so sánh các công cụ CI/CD phổ biến

Tiêu chí	Jenkins	GitLab CI/CD	GitHub Actions	ArgoCD
Mô hình triển khai	Self-Hosted	Self-Hosted hoặc Cloud SaaS	Cloud SaaS (hỗ trợ Self Runner)	Kubernetes Native

Tiêu chí	Jenkins	GitLab CI/CD	GitHub Actions	ArgoCD
Cấu hình Pipeline	Groovy (Jenkinsfile)	YAML (<code>.gitlab-ci.yml</code>)	YAML (<code>.github/workflows</code>)	K8s Manifests / Helm / Kustomize
Mức độ phức tạp	Cao	Trung bình	Thấp	Trung bình - Cao
Điểm mạnh	Khả năng mở rộng qua Plugins	Tích hợp All-in-one DevSecOps	Cộng đồng Marketplace rộng lớn	Quản lý đồng bộ GitOps cho K8s
Phù hợp nhất cho	Hệ thống Legacy lớn, On-Premise	Doanh nghiệp dùng trọn bộ GitLab	Dự án đặt trên GitHub	Ứng dụng chạy trên Kubernetes

Phần 4: Công cụ hỗ trợ Test-Harness Engineering

Hệ thống Test-Harness được chia thành nhiều lớp công cụ khác nhau tùy thuộc vào vị trí và mục đích kiểm thử.

4.1 Unit Testing Frameworks (Lớp nền tảng)

Unit test yêu cầu các framework chạy cực nhanh, có khả năng cô lập các hàm/phương thức để kiểm tra tính đúng đắn của logic code.

- **Vitest / Jest (Hệ sinh thái JavaScript/TypeScript):**
 - *Jest*: Lâu đời, do Facebook phát triển, đầy đủ tính năng mock, snapshot testing đi kèm.
 - *Vitest*: Sinh sau đẻ muộn nhưng cực kỳ nhanh, tận dụng tối đa sức mạnh của Vite, tương thích hoàn toàn với API của Jest, phù hợp cho các dự án React, Vue, Svelte hiện đại.
- **PyTest (Python):**
 - Framework viết test bằng Python cực kỳ mạnh mẽ, cú pháp ngắn gọn (sử dụng câu lệnh `assert` thuần túy thay vì các hàm helper phức tạp), hỗ trợ cơ chế `fixture` rất linh hoạt để chuẩn bị dữ liệu test.
- **JUnit 5 (Java):**
 - Tiêu chuẩn vàng cho thế giới Java, hỗ trợ chạy song song, viết test có tham số hóa (parameterized tests) và kiến trúc modular hóa tốt.

4.2 Integration & API Testing Tools

Kiểm thử sự tương tác giữa ứng dụng và các dịch vụ API bên ngoài hoặc các thành phần nội bộ.

- **Postman / Newman:**
 - *Postman*: Công cụ UI thiết kế và gọi API phổ biến.
 - *Newman*: Phiên bản dòng lệnh (CLI) của Postman, cho phép chạy các bộ sưu tập API test (Collections) trực tiếp trong CI/CD pipeline.
- **REST-Assured (Java):**
 - Thư viện Java chuyên biệt cung cấp cú pháp theo phong cách BDD (Given-When-Then) để kiểm thử các dịch vụ RESTful API một cách trực quan.

4.3 End-to-End (E2E) Testing Frameworks

Kiểm thử toàn bộ luồng nghiệp vụ bằng cách giả lập hành vi người dùng trên trình duyệt hoặc thiết bị di động thực tế.

- **Playwright (Microsoft):**
 - **Công cụ hàng đầu hiện nay.** Hỗ trợ kiểm thử đa trình duyệt (Chromium, WebKit, Firefox) chỉ với một API duy nhất.
 - **Đặc điểm nổi bật:** Tốc độ chạy rất nhanh nhờ kiến trúc kết nối qua giao thức gRPC/Websocket trực tiếp tới trình duyệt, tự động đợi phần tử hiển thị (Auto-waiting), chụp ảnh màn hình/video tự động khi lỗi, hỗ trợ chạy song song cực kỳ tốt.
- **Cypress:**
 - Chạy trực tiếp bên trong trình duyệt nên việc debug cực kỳ dễ dàng bằng DevTools. Tuy nhiên, Cypress gặp hạn chế khi cần kiểm thử đa tab, đa cửa sổ hoặc tương tác chéo tên miền (cross-origin).
- **Selenium WebDriver:**
 - "Lão làng" của ngành tự động hóa. Ưu điểm là hỗ trợ hầu hết mọi ngôn ngữ lập trình (Java, Python, C#, JS) nhưng tốc độ chạy chậm, cấu hình phức tạp và dễ bị lỗi Flaky do không có cơ chế tự đợi thông minh như Playwright.
- **Appium:**
 - Thư viện mở rộng từ Selenium dành riêng cho việc kiểm thử tự động ứng dụng di động (Native, Hybrid, Mobile Web) trên iOS và Android.

4.4 Mocking, Stubbing & Service Virtualization

Giúp hệ thống Test Harness độc lập với các dịch vụ bên ngoài, loại bỏ rủi ro môi trường API bên thứ ba bị sập hoặc trả về dữ liệu không mong muốn.

- **WireMock:**

- Công cụ giả lập HTTP API mạnh mẽ nhất. Cho phép bạn định nghĩa các phản hồi (response) giả lập dựa trên URL, Header, Body của request đầu vào. Có thể chạy độc lập dưới dạng Docker container.
- **Mockoon:**
 - Ứng dụng desktop trực quan giúp tạo nhanh các mock server cục bộ mà không cần viết cấu hình phức tạp.
- **Sinon.js / Mockito:**
 - Thư viện chuyên dụng để tạo spy, stub, mock cho các hàm, class nội bộ trong code logic tương ứng với JavaScript và Java.

4.5 Performance & Load Testing Tools

Kiểm tra khả năng chịu tải, giới hạn hiệu năng và độ ổn định của hệ thống dưới áp lực lớn.

- **k6 (Grafana Labs):**
 - **Khuyên dùng cho CI/CD.** Viết kịch bản kiểm thử bằng JavaScript/TypeScript nhưng thực thi bằng lõi Go hiệu năng cao. k6 tiêu tốn cực ít tài nguyên hệ thống (RAM/CPU) so với các công cụ khác và xuất kết quả trực tiếp ra các định dạng mà CI/CD dễ dàng phân tích hoặc đẩy về Prometheus/Grafana.
- **Apache JMeter:**
 - Công cụ kiểm thử tải truyền thống với giao diện đồ họa. Rất mạnh mẽ với nhiều tính năng nâng cao, nhưng khó tích hợp mượt mà vào luồng CI/CD dưới dạng code (IaC - Infrastructure as Code).
- **Locust:**
 - Công cụ kiểm thử tải viết bằng Python, cho phép định nghĩa hành vi người dùng bằng code Python thuần túy, dễ dàng mở rộng chạy phân tán (Distributed load testing).

4.6 Test Reporting & Analytics Dashboards

Nơi tổng hợp dữ liệu kiểm thử từ nhiều nguồn để các bên liên quan (Developer, QA, Product Owner) đánh giá chất lượng sản phẩm.

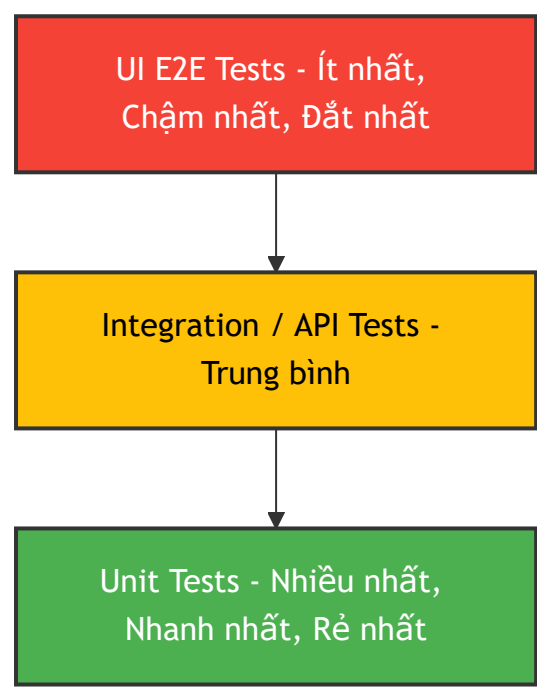
- **Allure Report:**
 - Trình tạo báo cáo HTML đa ngôn ngữ đẹp mắt, cung cấp biểu đồ trực quan về tỉ lệ pass/fail, thời gian chạy của từng test case, đính kèm được ảnh chụp lỗi và log chi tiết. Tích hợp dễ dàng vào Jenkins, GitLab CI.
 - **ReportPortal:**
 - Một nền tảng quản lý kết quả kiểm thử sử dụng AI/ML để phân tích log lỗi. Nó có khả năng tự động phân loại lỗi kiểm thử (như lỗi do Product Bug, Automation Bug, hay Môi trường mạng bị lag), giúp giảm thời gian phân tích kết quả thử công của đội ngũ QA.
-

Phần 5: Chiến lược tích hợp Test-Harness vào CI/CD Pipeline

Thiết kế một Test Harness mạnh mẽ là chưa đủ, việc tích hợp nó vào CI/CD pipeline sao cho cân bằng giữa **tốc độ phản hồi** và **độ tin cậy** mới là yếu tố quyết định sự thành bại.

5.1 Áp dụng Kim tự tháp Kiểm thử (Test Pyramid) vào Pipeline

Để tối ưu hóa chi phí và tốc độ, quy trình kiểm thử trong pipeline nên tuân theo mô hình Kim tự tháp kiểm thử:



- **Tầng đáy (Unit Tests):** Chạy ngay tại bước Build của CI. Nếu Unit test thất bại, dừng pipeline ngay lập tức. Thời gian chạy chỉ nên kéo dài từ vài giây đến tối đa 3-5 phút.
- **Tầng giữa (API/Integration Tests):** Chạy sau khi ứng dụng được build và đóng gói thành công. Xác minh tính đúng đắn khi các dịch vụ giao tiếp với nhau hoặc kết nối Database. Thời gian chạy khuyến nghị: 5-15 phút.
- **Tầng đỉnh (E2E Tests):** Chỉ chạy sau khi đã deploy lên môi trường Staging. Vì các bài test UI chạy rất chậm và dễ bị ảnh hưởng bởi môi trường, nên giới hạn số lượng kịch bản (chỉ kiểm thử luồng nghiệp vụ cốt lõi - Happy Paths).

5.2 Kỹ thuật chạy kiểm thử song song (Test Parallelization)

Khi số lượng test case tăng lên, việc chạy tuần tự sẽ khiến CI/CD pipeline bị nghẽn.

- **Giải pháp:** Phân chia các file test để chạy song song trên nhiều luồng (CPU Threads) của cùng một máy agent, hoặc phân phối sang nhiều máy agent CI khác nhau (Sharding).

- **Ví dụ với Playwright:** Playwright hỗ trợ sharding mặc định thông qua dòng lệnh:
`npx playwright test --shard=1/3` , `npx playwright test --shard=2/3` ,
`npx playwright test --shard=3/3` để chia đều bộ test suite ra 3 máy chạy song song trong GitHub Actions.

5.3 Xử lý lỗi không ổn định (Flaky Tests)

Flaky Test là bài test có lúc pass, có lúc fail trên cùng một phiên bản mã nguồn mà không có sự thay đổi nào. Lỗi này làm mất lòng tin của lập trình viên vào hệ thống CI/CD.

- **Nguyên nhân:** Bất đồng bộ trong mã nguồn (Race conditions), độ trễ mạng, dữ liệu test bị trùng lặp, hoặc tài nguyên máy chủ CI bị quá tải.
- **Chiến lược xử lý trong Test Harness:**
 - Auto-retry:** Cấu hình Test Runner tự động chạy lại ca kiểm thử bị fail tối đa 2-3 lần trước khi đánh dấu là thất bại hoàn toàn.
 - Quarantine (Cách ly):** Gắn nhãn `@flaky` hoặc `@quarantine` để loại bỏ các bài test không ổn định ra khỏi luồng chặn pipeline chính và đưa vào một luồng riêng để đội ngũ QA sửa chữa.
 - Strict Timeout:** Thiết lập thời gian chờ tối đa rõ ràng cho từng tương tác thay vì chờ đợi vô hạn.

5.4 Shift-Left & Shift-Right Testing

- **Shift-Left (Dịch chuyển về bên trái):** Đưa hoạt động kiểm thử lên sớm nhất có thể trong chu kỳ phát triển. Ví dụ: Lập trình viên chạy linting, quét tĩnh (SAST) ngay trên máy cá nhân thông qua Git Hooks (sử dụng thư viện `husky` và `lint-staged`) trước khi push code lên repo.
 - **Shift-Right (Dịch chuyển về bên phải):** Thực hiện kiểm thử trực tiếp trên môi trường Production bằng các kỹ thuật nâng cao như:
 - *Canary Deployment:* Triển khai phiên bản mới cho 5% lượng người dùng thực tế và giám sát lỗi trước khi triển khai toàn bộ.
 - *Synthetic Monitoring:* Chạy các kịch bản test tự động định kỳ (ví dụ cứ 15 phút một lần) trực tiếp trên production để đảm bảo hệ thống vẫn hoạt động trơn tru.
-

Phần 6: Xu hướng tương lai của CI/CD & Test-Harness Engineering

6.1 Môi trường kiểm thử tạm thời (Ephemeral/Dynamic Environments)

Thay vì dùng chung một môi trường Staging cố định (dễ dẫn đến việc xung đột dữ liệu test của nhiều lập trình viên), xu hướng hiện nay là sử dụng hạ tầng động.

- **Cách thức:** Khi một lập trình viên mở một Pull Request, CI pipeline sẽ tự động khởi tạo một môi trường biệt lập mới hoàn toàn (thông qua Kubernetes Namespace hoặc Docker Compose) dành riêng cho nhánh code đó.
- Hệ thống Test Harness sẽ chạy toàn bộ các bài test E2E trên môi trường này. Khi PR được merge hoặc close, môi trường động này sẽ tự động bị xóa bỏ để tiết kiệm tài nguyên.

6.2 Ứng dụng AI/ML trong Kiểm thử tự động (AI-Driven Testing)

- **Tự động sửa lỗi test script (Self-healing tests):** Khi cấu trúc DOM của trang web thay đổi (ví dụ nút "Đăng nhập" đổi ID hoặc class), các công cụ AI hỗ trợ sẽ tự động phân tích và tìm ra phần tử thay thế mà không làm gãy bài test.
- **Tự động tạo ca kiểm thử:** Phân tích mã nguồn và lịch sử lỗi để sinh ra các kịch bản kiểm thử bao phủ các vùng code quan trọng hoặc có nguy cơ lỗi cao nhất.

6.3 GitOps & Infrastructure as Code (IaC) cho Hạ tầng Kiểm thử

Toàn bộ cấu hình hệ thống Test Harness (bao gồm database giả lập, cấu hình mạng, các biến môi trường) được định nghĩa dưới dạng mã nguồn (Infrastructure as Code - IaC) bằng Terraform, Ansible hoặc Docker Compose. Việc này đảm bảo môi trường kiểm thử luôn nhất quán, có thể tái tạo ở bất kỳ đâu chỉ bằng một câu lệnh duy nhất.